

CSS

Flexbox & Transitions

Table of Contents

Flexbox Layout	4
CSS Flexbox Background	4
Flexbox basics and terminology.....	4
Properties for the Parent (flex container)	5
display	5
flex-direction	6
flex-wrap	7
flex-flow	7
justify-content	8
align-items	10
align-content	11
gap, row-gap, column-gap	13
Properties for the Children (flex items)	14
order	14
flex-grow	14
flex-shrink	15
flex-basis	15
flex	15
align-self	16
Prefixing Flexbox	16
CSS Flexbox Examples.....	17
Example1	19
Example2	21
Cheatsheet.....	23
flex container attributes	23
flex items attributes	24
More reading resources	25
Flexbox and absolute positioning	25

Adaptive Photo layout	25
Using Flexbox and text ellipsis together	25
Designing A Product Page Layout with Flexbox	25
Balancing on a Pivot with Flexbox.....	25
Useful Flexbox Technique: Alignment Shifting Wrapping	25
Flexbox and Truncated Text	25
Filling the Space in the Last Row with Flexbox	25
CSS Transitions in s&box — Walkthrough Glossary	26
1. The transition shorthand and its four sub-properties	26
transition (shorthand)	26
transition-property	26
transition-duration	26
transition-timing-function	26
transition-delay	26
&:hover (pseudo-class)	26
& (SCSS parent reference)	26
2. Longhand form.....	27
Lining up the lists	27
3. Multiple transitions in one shorthand.....	27
Comma-separated tuples	27
4. Timing functions.....	27
linear	27
ease.....	27
ease-in	27
ease-out	27
ease-in-out	27
cubic-bezier(x1, y1, x2, y2)	27
5. ` and:outro` — the s&box-only pseudo-classes.....	28
:intro	28
:outro	28
Panel.Delete() (C# method)	28
6. State classes — the bread-and-butter pattern	28
Panel.SetClass(string name, bool on) (C# method).....	28
Why not just re-render?.....	28
7. Razor-driven inline style.....	29

Inline style="..." attribute.....	29
Why this pattern matters in s&box	29
8. Asymmetric enter/exit	29
Different transitions per state.....	29
9. SCSS variables for design tokens.....	29
\$variable: value;	29
Why these, not CSS custom properties?	29
10. Cost matrix — cheap vs expensive properties	30
GPU-composited (cheap)	30
Paint-only (medium)	30
Layout-triggering (expensive).....	30
Rule of thumb	30
11. Sound on enter/exit	30
sound-in / sound-out (s&box custom CSS) ??	30
12. Things you cannot transition.....	30
Discrete property changes	30
auto values	31
The "fake hide" pattern.....	31
Mental model in one sentence	31

Flexbox Layout



Chris Coyier on Apr 8, 2013

This complete guide explains everything about flexbox, focusing on all the different possible properties for the parent element (the flex container) and the child elements (the flex items). It also includes history, demos, patterns, and a browser support chart.

CSS Flexbox Background

The Flexbox Layout (Flexible Box) module provides a more efficient way to lay out, align and distribute space among items in a container, even when their size is unknown and/or dynamic (thus the word “flex”).

The main idea behind the flex layout is to give the container the ability to alter its items’ width/height (and order) to best fill the available space (mostly to accommodate to all kind of display devices and screen sizes). A flex container expands items to fill available free space or shrinks them to prevent overflow.

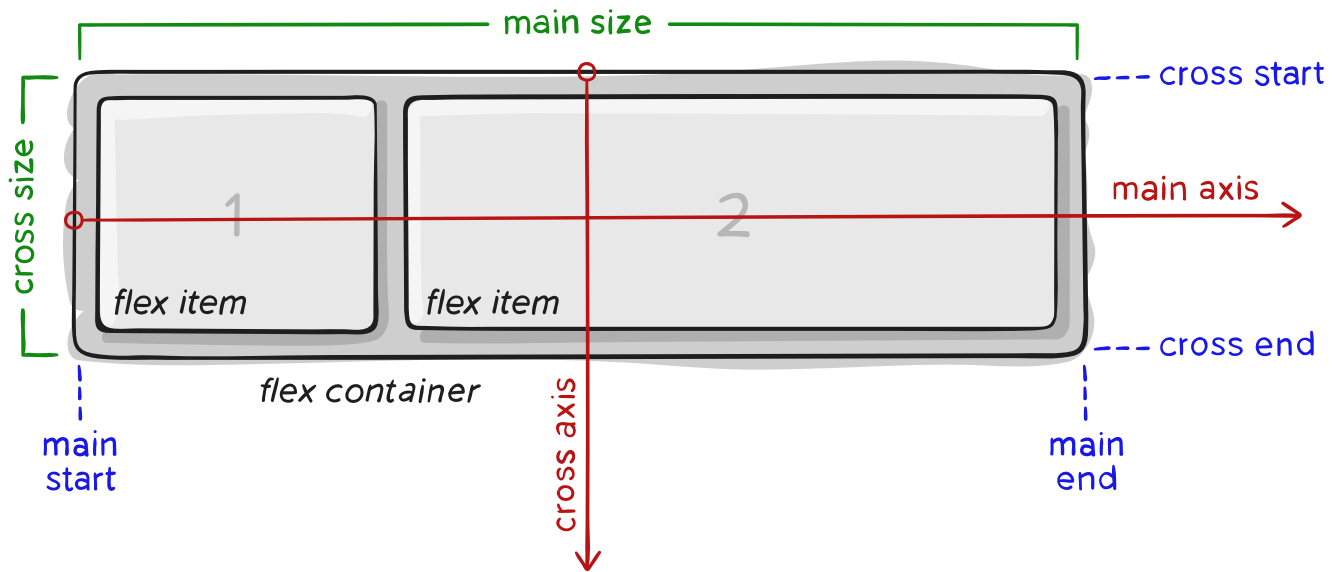
Most importantly, the flexbox layout is direction-agnostic as opposed to the regular layouts (block which is vertically-based and inline which is horizontally-based). While those work well for pages, they lack flexibility (no pun intended) to support large or complex applications (especially when it comes to orientation changing, resizing, stretching, shrinking, etc.).

Note: Flexbox layout is most appropriate to the components of an application, and small-scale layouts, while the [CSS Grid](#) layout is intended for larger scale layouts.

Flexbox basics and terminology

Since flexbox is a whole module and not a single property, it involves a lot of things including its whole set of properties. Some of them are meant to be set on the container (parent element, known as “flex container”) whereas the others are meant to be set on the children (said “flex items”).

If “regular” layout is based on both block and inline flow directions, the flex layout is based on “flex-flow directions”. Please have a look at this figure from the specification, explaining the main idea behind the flex layout.



Items will be laid out following either the main axis (from main-start to main-end) or the cross axis (from cross-start to cross-end).

- **main axis** – The main axis of a flex container is the primary axis along which flex items are laid out. Beware, it is not necessarily horizontal; it depends on the flex-direction property (see below).
- **main-start | main-end** – The flex items are placed within the container starting from main-start and going to main-end.
- **main size** – A flex item’s width or height, whichever is in the main dimension, is the item’s main size. The flex item’s main size property is either the ‘width’ or ‘height’ property, whichever is in the main dimension.
- **cross axis** – The axis perpendicular to the main axis is called the cross axis. Its direction depends on the main axis direction.
- **cross-start | cross-end** – Flex lines are filled with items and placed into the container starting on the cross-start side of the flex container and going toward the cross-end side.
- **cross size** – The width or height of a flex item, whichever is in the cross dimension, is the item’s cross size. The cross size property is whichever of ‘width’ or ‘height’ that is in the cross dimension.

Properties for the Parent (flex container)

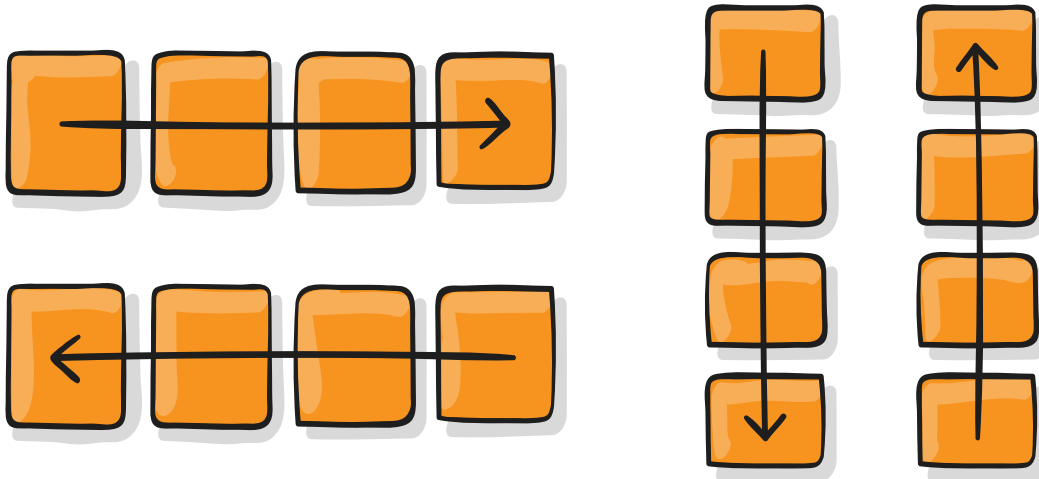
display

This defines a flex container; inline or block depending on the given value. It enables a flex context for all its direct children.

```
.container {
  display: flex; /* or inline-flex */
}
```

Note that CSS columns have no effect on a flex container.

flex-direction

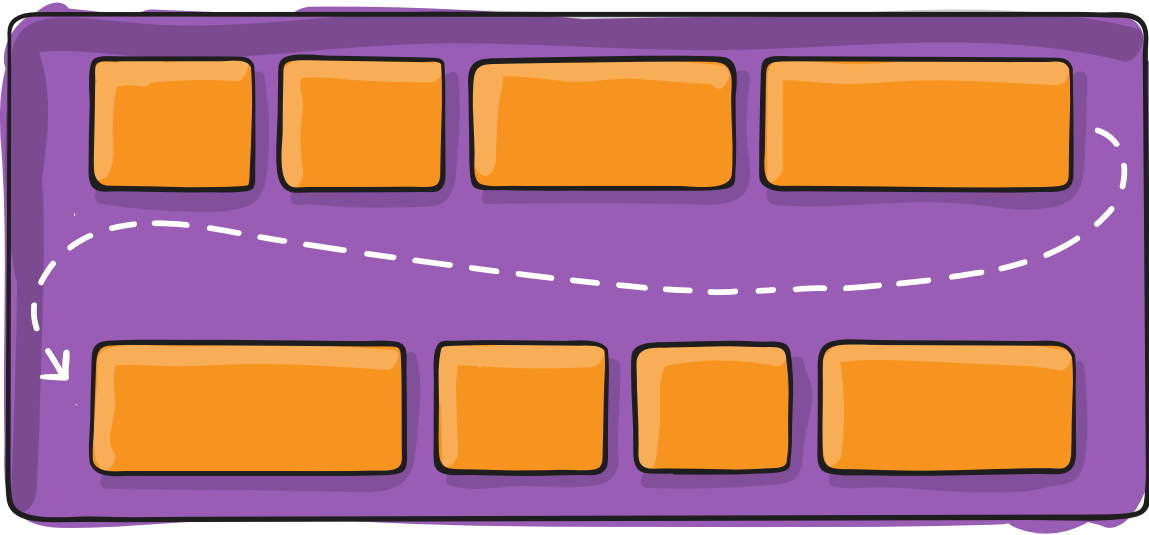


This establishes the main-axis, thus defining the direction flex items are placed in the flex container. Flexbox is (aside from optional wrapping) a single-direction layout concept. Think of flex items as primarily laying out either in horizontal rows or vertical columns.

```
.container {
  flex-direction: row | row-reverse | column | column-reverse;
}
```

- row (default): left to right in ltr; right to left in rtl
- row-reverse: right to left in ltr; left to right in rtl
- column: same as row but top to bottom
- column-reverse: same as row-reverse but bottom to top

flex-wrap



By default, flex items will all try to fit onto one line. You can change that and allow the items to wrap as needed with this property.

```
.container {  
  flex-wrap: nowrap | wrap | wrap-reverse;  
}
```

- nowrap (default): all flex items will be on one line
- wrap: flex items will wrap onto multiple lines, from top to bottom.
- wrap-reverse: flex items will wrap onto multiple lines from bottom to top.

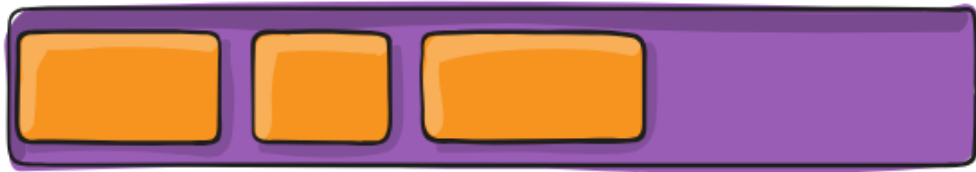
There are some [visual demos of flex-wrap here](#).

flex-flow

This is a shorthand for the flex-direction and flex-wrap properties, which together define the flex container's main and cross axes. The default value is row nowrap.

```
.container {  
  flex-flow: column wrap;  
}
```

flex-start



flex-end



center



space-between



space-around



space-evenly



This defines the alignment along the main axis. It helps distribute extra free space leftover when either all the flex items on a line are inflexible, or are flexible but have reached their maximum size. It also exerts some control over the alignment of items when they overflow the line.


```
.container {  
  justify-content: flex-start | flex-end | center | space-between | space-around | space-evenly | start | end | left | right ... + safe | unsafe;  
}
```

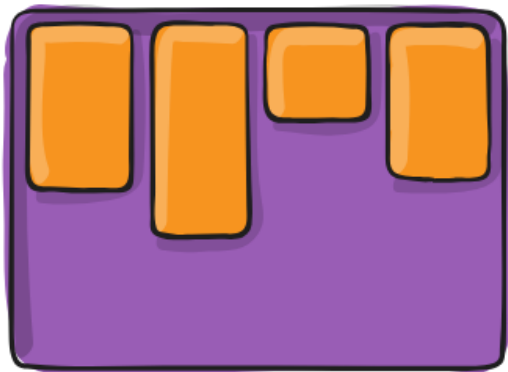
- flex-start (default): items are packed toward the start of the flex-direction.
- flex-end: items are packed toward the end of the flex-direction.
- start: items are packed toward the start of the writing-mode direction.
- end: items are packed toward the end of the writing-mode direction.
- left: items are packed toward left edge of the container, unless that doesn't make sense with the flex-direction, then it behaves like start.
- right: items are packed toward right edge of the container, unless that doesn't make sense with the flex-direction, then it behaves like end.
- center: items are centered along the line
- space-between: items are evenly distributed in the line; first item is on the start line, last item on the end line
- space-around: items are evenly distributed in the line with equal space around them. Note that visually the spaces aren't equal, since all the items have equal space on both sides. The first item will have one unit of space against the container edge, but two units of space between the next item because that next item has its own spacing that applies.
- space-evenly: items are distributed so that the spacing between any two items (and the space to the edges) is equal.

Note that that browser support for these values is nuanced. For example, space-between never got support from some versions of Edge, and start/end/left/right aren't in Chrome yet. MDN [has detailed charts](#). The safest values are flex-start, flex-end, and center.

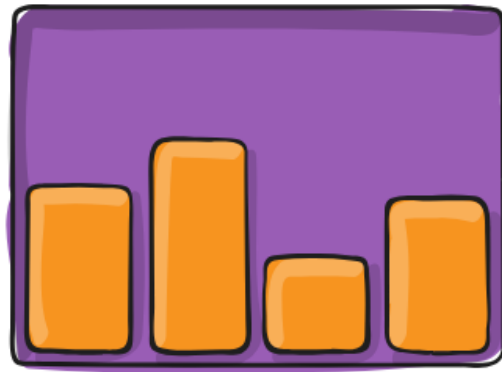
There are also two additional keywords you can pair with these values: safe and unsafe. Using safe ensures that however you do this type of positioning, you can't push an element such that it renders off-screen (e.g. off the top) in such a way the content can't be scrolled too (called "data loss").

align-items

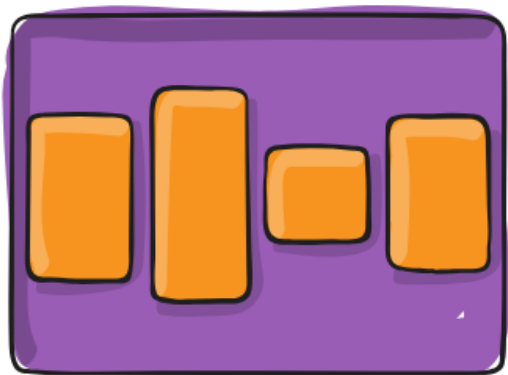
flex-start



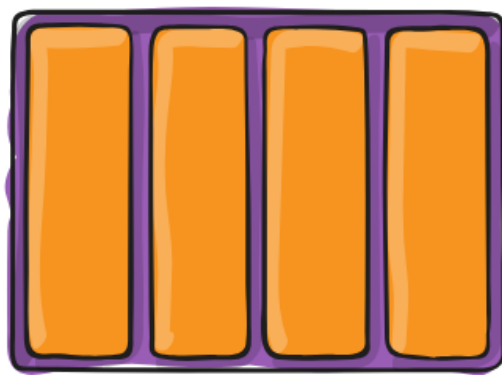
flex-end



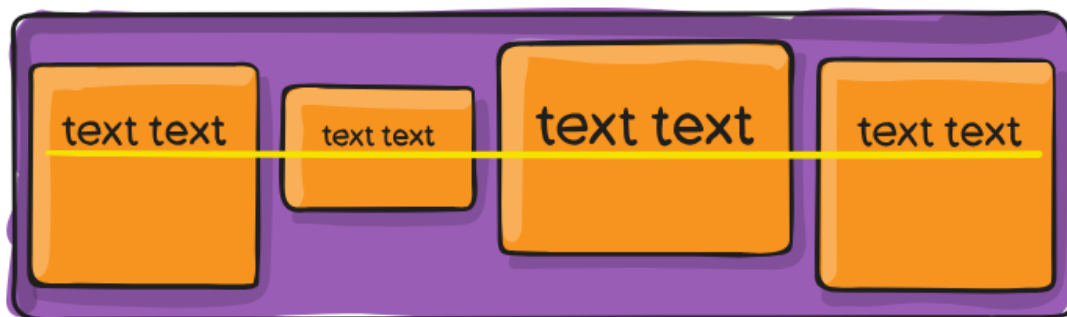
center



stretch



baseline



This defines the default behavior for how flex items are laid out along the **cross axis** on the current line. Think of it as the justify-content version for the cross-axis (perpendicular to the main-axis).

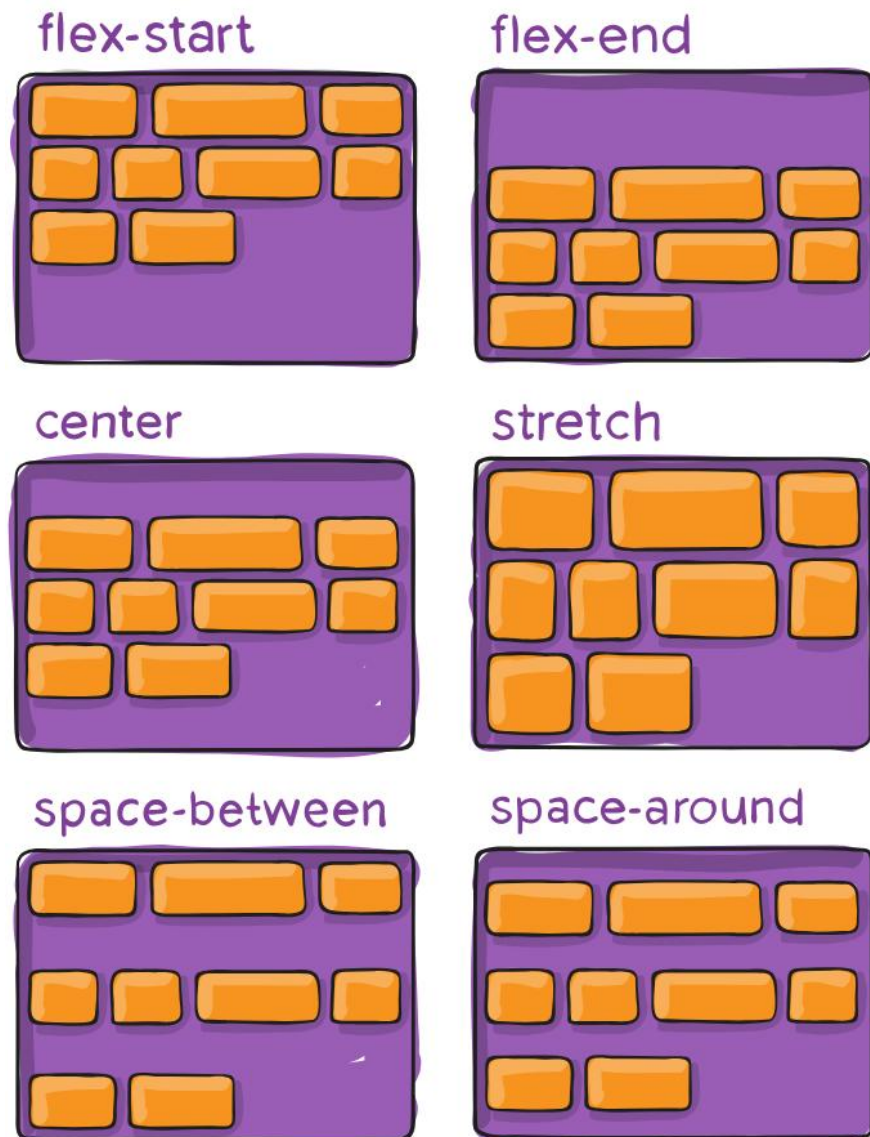
```
.container {  
  align-items: stretch | flex-start | flex-end | center | baseline | first baseline | last baseline | start | end | self-start | self-end + ... safe | unsafe;  
}
```

- stretch (default): stretch to fill the container (still respect min-width/max-width)

- `flex-start` / `start` / `self-start`: items are placed at the start of the cross axis. The difference between these is subtle, and is about respecting the flex-direction rules or the writing-mode rules.
- `flex-end` / `end` / `self-end`: items are placed at the end of the cross axis. The difference again is subtle and is about respecting flex-direction rules vs. writing-mode rules.
- `center`: items are centered in the cross-axis
- `baseline`: items are aligned such as their baselines align

The safe and unsafe modifier keywords can be used in conjunction with all the rest of these keywords (although note [browser support](#)), and deal with helping you prevent aligning elements such that the content becomes inaccessible.

align-content



This aligns a flex container's lines within when there is extra space in the cross-axis, similar to how justify-content aligns individual items within the main-axis.

Note: This property only takes effect on multi-line flexible containers, where flex-wrap is set to either wrap or wrap-reverse). A single-line flexible container (i.e. where flex-wrap is set to its default value, no-wrap) will not reflect align-content.

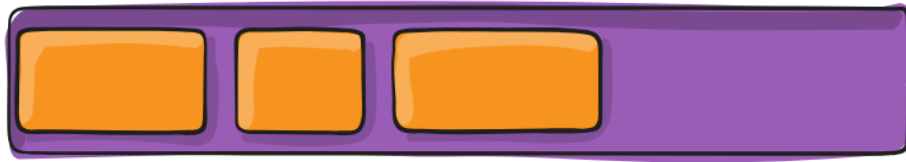
```
.container {  
  align-content: flex-start | flex-end | center | space-between | space-around | space-evenly | stretch | start | end | baseline | first baseline | last baseline + ... safe | unsafe;  
}
```

- normal (default): items are packed in their default position as if no value was set.
- flex-start / start: items packed to the start of the container. The (more supported) flex-start honors the flex-direction while start honors the writing-mode direction.
- flex-end / end: items packed to the end of the container. The (more support) flex-end honors the flex-direction while end honors the writing-mode direction.
- center: items centered in the container
- space-between: items evenly distributed; the first line is at the start of the container while the last one is at the end
- space-around: items evenly distributed with equal space around each line
- space-evenly: items are evenly distributed with equal space around them
- stretch: lines stretch to take up the remaining space

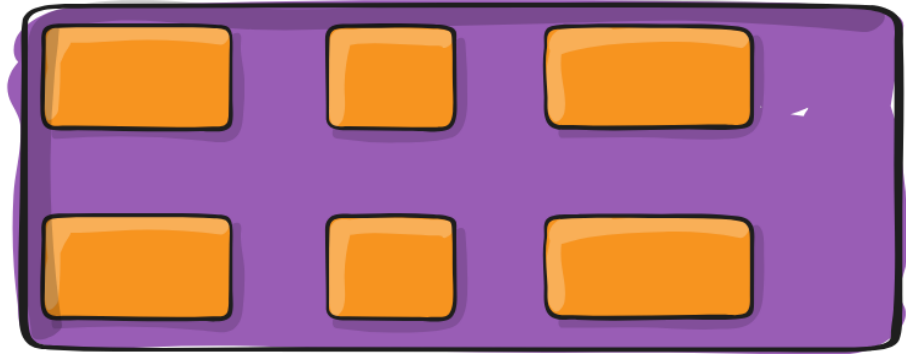
The safe and unsafe modifier keywords can be used in conjunction with all the rest of these keywords (although note [browser support](#)), and deal with helping you prevent aligning elements such that the content becomes inaccessible.

gap, row-gap, column-gap

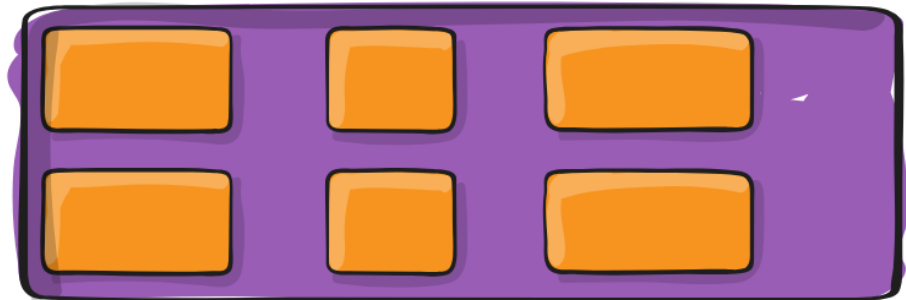
gap: 10px



gap: 30px



gap: 10px 30px



The `gap` property explicitly controls the space between flex items. It applies that spacing *only between items* not on the outer edges.

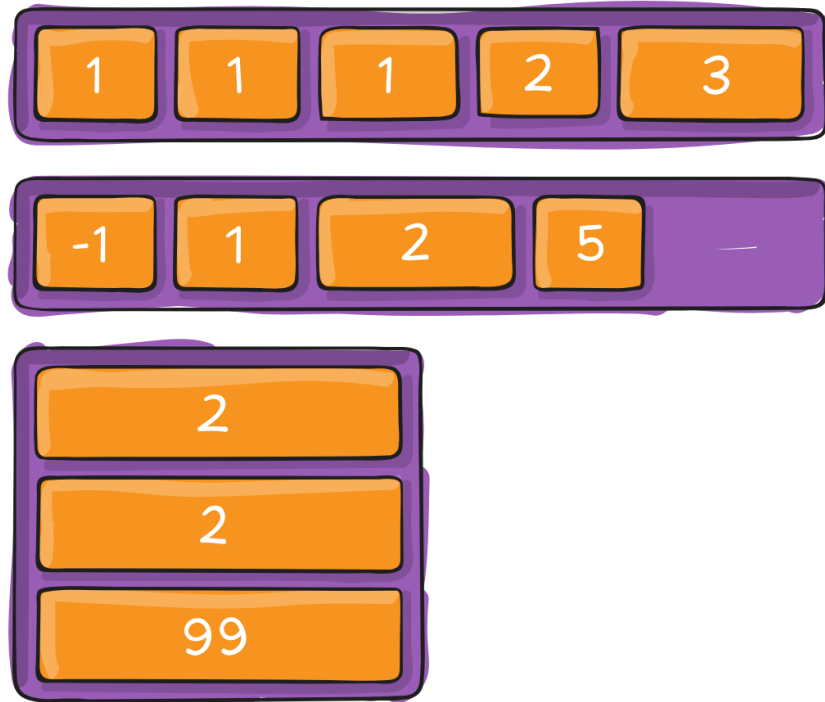
```
.container {  
  display: flex;  
  ...  
  gap: 10px;  
  gap: 10px 20px; /* row-gap column gap */  
  row-gap: 10px;  
  column-gap: 20px;  
}
```

The behavior could be thought of as a *minimum* gutter, as if the gutter is bigger somehow (because of something like `justify-content: space-between;`) then the gap will only take effect if that space would end up smaller.

It is not exclusively for flexbox, gap works in grid and multi-column layout as well.

Properties for the Children (flex items)

order

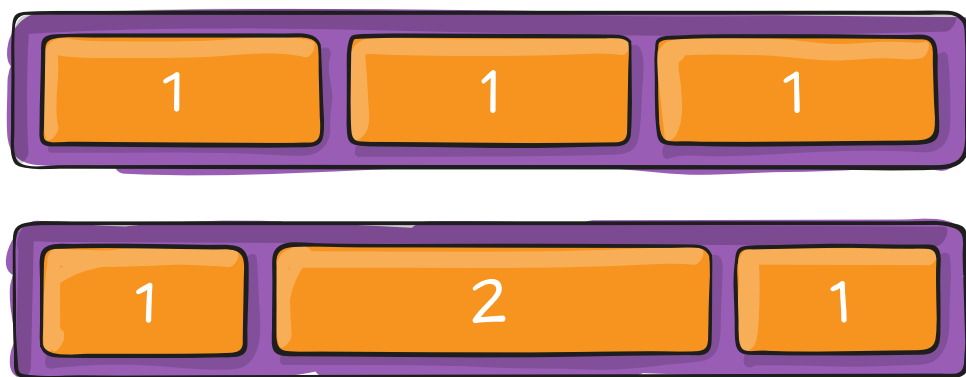


By default, flex items are laid out in the source order. However, the `order` property controls the order in which they appear in the flex container.

```
.item {  
  order: 5; /* default is 0 */  
}
```

Items with the same order revert to source order.

flex-grow



This defines the ability for a flex item to grow if necessary. It accepts a unitless value that serves as a proportion. It dictates what amount of the available space inside the flex container the item should take up.

If all items have `flex-grow` set to 1, the remaining space in the container will be distributed equally to all children. If one of the children has a value of 2, that child would take up twice as much of the space as either one of the others (or it will try, at least).

```
.item {  
  flex-grow: 4; /* default 0 */  
}
```

Negative numbers are invalid.

flex-shrink

This defines the ability for a flex item to shrink if necessary.

```
.item {  
  flex-shrink: 3; /* default 1 */  
}
```

Negative numbers are invalid.

flex-basis

This defines the default size of an element before the remaining space is distributed. It can be a length (e.g. 20%, 5rem, etc.) or a keyword. The `auto` keyword means “look at my width or height property” (which was temporarily done by the `main-size` keyword until deprecated). The `content` keyword means “size it based on the item’s content” – this keyword isn’t well supported yet, so it’s hard to test and harder to know what its brethren `max-content`, `min-content`, and `fit-content` do.

```
.item {  
  flex-basis: | auto; /* default auto */  
}
```

If set to `0`, the extra space around content isn’t factored in. If set to `auto`, the extra space is distributed based on its `flex-grow` value. [See this graphic.](#)

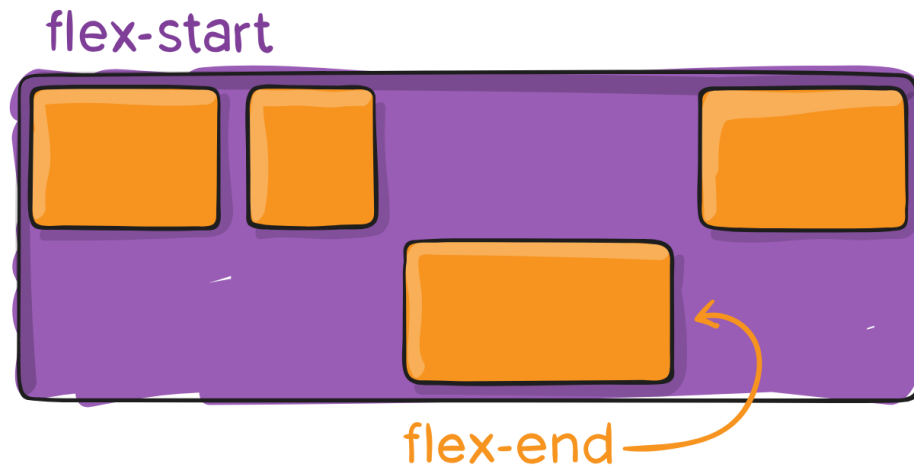
flex

This is the shorthand for `flex-grow`, `flex-shrink` and `flex-basis` combined. The second and third parameters (`flex-shrink` and `flex-basis`) are optional. The default is `0 1 auto`, but if you set it with a single number value, like `flex: 5;`, that changes the `flex-basis` to `0%`, so it’s like setting `flex-grow: 5;` `flex-shrink: 1;` `flex-basis: 0%;`.

```
.item {  
  flex: none | [ <'flex-grow'> <'flex-shrink'>? || <'flex-basis'> ]  
}
```

It is recommended that you use this shorthand property rather than set the individual properties. The shorthand sets the other values intelligently.

align-self



This allows the default alignment (or the one specified by `align-items`) to be overridden for individual flex items.

Please see the `align-items` explanation to understand the available values.

```
.item {  
  align-self: auto | flex-start | flex-end | center | baseline | stretch;  
}
```

Note that `float`, `clear` and `vertical-align` have no effect on a flex item.

Prefixing Flexbox

CSS Flexbox is widely supported in all modern browsers, but requires some vendor prefixing to support the legacy browsers, like Internet Explorer (which has been discontinued for a while now). It doesn't just include prepending properties with the vendor prefix, but there are actually entirely different property and value names. This is because the Flexbox spec has changed over time, creating an “old”, “tweener”, and “new” versions.

Perhaps the best way to handle this is to write in the new (and final) syntax and run your CSS through [Autoprefixer](#), which handles the fallbacks very well.

Alternatively, here's a Sass `@mixin` to help with some of the prefixing, which also gives you an idea of what kind of things need to be done:

```
@mixin flexbox() {  
  display: -webkit-box;  
  display: -moz-box;  
  display: -ms-flexbox;
```



```

display: -webkit-flex;
display: flex;
}

@mixin flex($values) {
  -webkit-box-flex: $values;
  -moz-box-flex: $values;
  -webkit-flex: $values;
  -ms-flex: $values;
  flex: $values;
}

@mixin order($val) {
  -webkit-box-ordinal-group: $val;
  -moz-box-ordinal-group: $val;
  -ms-flex-order: $val;
  -webkit-order: $val;
  order: $val;
}

.wrapper {
  @include flexbox();
}

.item {
  @include flex(1 200px);
  @include order(2);
}

```

CSS Flexbox Examples

Let's start with a very very simple example, solving an almost daily problem: perfect centering. It couldn't be any simpler if you use CSS Flexbox.

```

.parent {
  display: flex;
  height: 300px; /* Or whatever */
}

.child {
  width: 100px; /* Or whatever */
  height: 100px; /* Or whatever */
  margin: auto; /* Magic! */
}

```

This relies on the fact a margin set to auto in a flex container absorb extra space. So setting a margin of auto will make the item perfectly centered in both axes.

Now let's use some more properties. Consider a list of 6 items, all with fixed dimensions, but can be auto-sized. We want them to be evenly distributed on the horizontal axis so that when we resize the browser, everything scales nicely, and without media queries.

```
<html>
  <head>
    <title>My Page</title>
    <link rel="stylesheet" href="UI.css">
  </head>
  <body>
    <ul>
      <li>I am list 0</li>
      <li>I am list 1</li>
      <li>I am list 2</li>
      <li>I am list 3</li>
      <li>I am list 4</li>
      <!-- <li>I am list 5</li>
      <li>I am list 7</li>
      <li>I am list 8</li>
      <li>I am list 9</li> -->
    </ul>
  </body>
</html>
```

```
ul {
  display: flex; /* We first create a flex layout context */
  /* Then we define the flow direction & if we allow the items to wrap
  * Remember this is the same as:
  * flex-direction: row;
  * flex-wrap: wrap;
  */
  flex-flow: row nowrap;
  list-style: none; /* kill the bullets */
  padding: 0; /* kill the default indent */
  margin: 0;
  background-color: blueviolet;
  align-items: baseline;
  height: 60%;
}

li{
  border: 1px solid white;
  padding: 5px;
  margin: 2px;
  background-color: aquamarine;
  border-radius: 2px;
}
```

Result



Example1

Let's try something else. Imagine we have a right-aligned navigation element on the very top of our website, but we want it to be centered on medium-sized screens and single-columned on small devices. Easy enough.

```
/* Large */
.navigation {
  display: flex;
  flex-flow: row wrap;
  /* This aligns items to the end line on main-axis */
  justify-content: flex-end;
}

/* Medium screens */
@media all and (max-width: 800px) {
  .navigation {
    /* When on medium sized screens, we center it by evenly distributing empty space around items */
    justify-content: space-around;
  }
}

/* Small screens */
@media all and (max-width: 500px) {
  .navigation {
    /* On small screens, we are no longer using row direction but column */
    flex-direction: column;
  }
}
```

Code:

```
<html>
  <head>
    <title>My Page</title>
    <link rel="stylesheet" href="UI.css">
  </head>
  <body>
```

```

        <ul>
          <ul class="navigation">
            <li><a href="#">Home</a></li>
            <li><a href="#">About</a></li>
            <li><a href="#">Products</a></li>
            <li><a href="#">Contact</a></li>
          </ul>
        </ul>
      </body>
</html>

```

```

.navigation {
  display: flex;
  flex-flow: row wrap;
  justify-content: flex-end;

  list-style: none;
  margin: 0;
  background: deepskyblue;
}

```

```

.navigation a {
  text-decoration: none;
  display: block;
  padding: 1em;
  color: white;
}

```

```

.navigation a:hover {
  background: #1565C0;
}

```

```

@media all and (max-width: 800px) {
  .navigation {
    justify-content: space-around;
  }
}

```

```

@media all and (max-width: 600px) {
  .navigation {
    flex-flow: column wrap;
    padding: 0;
  }
  .navigation a {
    text-align: center;
    padding: 10px;
    border-top: 1px solid rgba(255, 255, 255, 0.3);
    border-bottom: 1px solid rgba(0, 0, 0, 0.1);
  }
  .navigation li:last-of-type a {
    border-bottom: none;
  }
}

```

```
}  
}
```

Result:



Example2

Let's try something even better by playing with flex items flexibility! What about a mobile-first 3-columns layout with full-width header and footer. And independent from source order.

```
.wrapper {  
  display: flex;  
  flex-flow: row wrap;  
}  
  
/* We tell all items to be 100% width, via flex-basis */  
.wrapper > * {  
  flex: 1 100%;  
}  
  
/* We rely on source order for mobile-first approach  
 * in this case:  
 * 1. header  
 * 2. article  
 * 3. aside 1  
 * 4. aside 2  
 * 5. footer  
 */  
  
/* Medium screens */  
@media all and (min-width: 600px) {  
  /* We tell both sidebars to share a row */  
  .aside { flex: 1 auto; }  
}  
  
/* Large screens */  
@media all and (min-width: 800px) {  
  /* We invert order of first sidebar and main  
   * And tell the main element to take twice as much width as the other two sidebars  
   */  
  .main { flex: 3 0px; }  
  .aside-1 { order: 1; }  
  .main { order: 2; }  
  .aside-2 { order: 3; }  
  .footer { order: 4; }  
}
```

Code

```

<html>
  <head>
    <title>My Page</title>
    <link rel="stylesheet" href="UI.css">
  </head>
  <body>
    <div class="wrapper">
      <header class="header">Header</header>
      <article class="main">
        <p>Pellentesque habitant morbi tristique senectus et netus et malesuada fames
        ac turpis egestas. Vestibulum tortor quam, feugiat vitae, ultricies eget, tempor sit ame
        t, ante. Donec eu libero sit amet quam egestas semper. Aenean ultricies mi vitae est. Mau
        ris placerat eleifend leo.</p>
      </article>
      <aside class="aside aside-1">Aside 1</aside>
      <aside class="aside aside-2">Aside 2</aside>
      <footer class="footer">Footer</footer>
    </div>
  </body>
</html>

```

```

.wrapper {
  display: flex;
  flex-flow: row wrap;
  font-weight: bold;
  text-align: center;
}

.wrapper > * {
  padding: 10px;
  flex: 1 100%;
}

.header {
  background: tomato;
}

.footer {
  background: lightgreen;
}

.main {
  text-align: left;
  background: deepskyblue;
}

.aside-1 {
  background: gold;
}

.aside-2 {

```

```

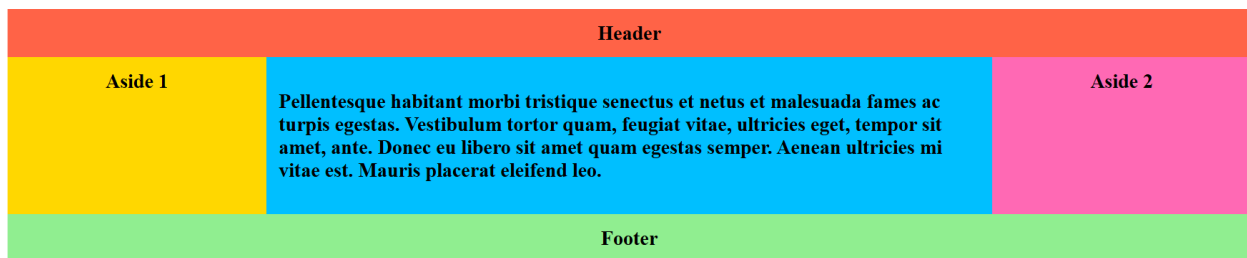
background: hotpink;
}

@media all and (min-width: 600px) {
  .aside { flex: 1 0 0; }
}

@media all and (min-width: 800px) {
  .main { flex: 3 0px; }
  .aside-1 { order: 1; }
  .main { order: 2; }
  .aside-2 { order: 3; }
  .footer { order: 4; }
}

body {
  padding: 2em;
}

```



Cheatsheet

flex container attributes

```

.container {
  /* Defines a flex container */
  display: flex; /* or inline-flex */

  /* Establishes direction aka main-axis flex items are placed in the flex container
  row (default): left to right */
  flex-direction: row | row-reverse | column | column-reverse;

  /* By default, flex items will all try to fit onto one line (nowrap). */
  flex-wrap: nowrap | wrap | wrap-reverse;

  /* shorthand for the flex-direction and flex-wrap properties, which together define the
  flex container's main and cross axes */
  flex-flow: column wrap;

  /* Defines the alignment along the main axis.
  flex-start (default): items are packed toward the start of the flex-direction. */
  justify-content: flex-start | flex-end | center | space-between | space-around |
  space-evenly | start | end | left | right + ... | safe | unsafe;
}

```

```

/* This defines the default behavior for how flex items are laid out along the cross
   axis on the current line. Think of it as the justify-content version for the cross-axis
   is (perpendicular to the main-axis).
   stretch (default): stretch to fill the container (still respect min-width/max-width)
*/
align-items: stretch | flex-start | flex-end | center | baseline | first baseline |
             last baseline | start | end | self-start | self-end + ... | safe | unsafe;

/* Aligns a flex container's lines within when there is extra space in the cross-axis,
   similar to how justify-content aligns individual items within the main-axis.
   normal (default): items are packed in their default position as if no value was set.
   Note: This property only takes effect on multi-line flexible containers, where flex-wrap
   is set to either wrap or wrap-reverse ). A single-line flexible container (i.e. where
   flex-wrap is set to its default value, no-wrap ) will not reflect align-content .*/
align-content: flex-start | flex-end | center | space-between | space-around | space-between
only | stretch | start | end | baseline | first baseline | last baseline + ... safe | unsafe;

/* gap property explicitly controls the space between flex items. It applies that spacing
   only between items not on the outer edges. */
gap: 10px;
gap: 10px 20px; /* row-gap column gap */
row-gap: 10px;
column-gap: 20px;
}

```

flex items attributes

```

.item{
/*Controls the order in which items appear in the flex container.*/
order: 5; /* default is 0 */

/* defines the ability for a flex item to grow if necessary. If all items have flex-grow
   w set to 1 , the remaining space in the container will be distributed equally to all children.
   If one of the children has a value of 2 , that child would take up twice as much of
   the space as either one of the others (or it will try, at least). */
flex-grow: 4; /* Default is 0*/

/*Defines the ability for a flex item to shrink if necessary -ve numbers are invalid. *
flex-shrink: 3; /* default 1 */

/* Defines the default size of an element before the remaining space is distributed. */
flex-basis: auto; /* default auto */

/* This is the shorthand for flex-grow, flex-shrink and flex-basis combined.
   It is recommended that you use this shorthand rather than set the individual properties.
   The default is 0 1 auto , but if you set it with a single number value, like flex: 5; ,
   that changes the flex-basis to 0%, so it's like setting flex-grow: 5; flex-shrink: 1; flex-basis: 0%; */
flex: none | flex-grow | flex-shrink | flex-basis /* Default: is 0 1 auto */
}

```



```
/* This allows the default alignment (or the one specified by align-items ) to be overr  
idden for individual flex items. */  
align-self: auto | flex-start | flex-end | center | baseline | stretch;  
}
```

More reading resources

Flexbox and absolute positioning

<https://css-tricks.com/flexbox-and-absolute-positioning/>

Adaptive Photo layout

<https://css-tricks.com/adaptive-photo-layout-with-flexbox/>

Using Flexbox and text ellipsis together

<https://css-tricks.com/using-flexbox-and-text-ellipsis-together/>

Designing A Product Page Layout with Flexbox

<https://css-tricks.com/designing-a-product-page-layout-with-flexbox/>

Balancing on a Pivot with Flexbox

<https://css-tricks.com/balancing-on-a-pivot-with-flexbox/>

Useful Flexbox Technique: Alignment Shifting Wrapping

<https://css-tricks.com/useful-flexbox-technique-alignment-shifting-wrapping/>

Flexbox and Truncated Text

<https://css-tricks.com/flexbox-truncated-text/>

Filling the Space in the Last Row with Flexbox

<https://css-tricks.com/filling-space-last-row-flexbox/>

CSS Transitions in s&box — Walkthrough Glossary

CSS transitions interpolate a property's value over time when it changes, instead of snapping. s&box implements a subset of standard web CSS plus a few engine-specific extensions for create/delete animations and sound. The entries below appear in the same order as `Transitions_annotated.scss`, so you can read both side-by-side.

1. The `transition` shorthand and its four sub-properties

`transition` (shorthand)

A CSS property that bundles up to four sub-properties into one declaration. The canonical form is `transition: <property> <duration> <timing-function> <delay>`. Order of times matters: the first time-value is duration, the second is delay.

`transition-property`

Names which CSS properties should animate when they change. Anything not listed snaps instantly. Comma-separated list. Special value `all` matches every changing property (convenient but a perf footgun in game UI).

`transition-duration`

How long the animation takes. Accepts seconds (s) or milliseconds (ms). One value, or one per property in `transition-property`.

`transition-timing-function`

The easing curve — i.e. *the rate of change over time*. Determines whether motion feels mechanical (linear), natural (ease-out), or playful (cubic-bezier with overshoot).

`transition-delay`

How long to wait after the property changes before starting the transition. Useful for staggering enter animations across siblings.

`&:hover` (pseudo-class)

Applied while the cursor is over the element. **Only fires if `pointer-events: all` is set somewhere up the ancestor chain.** s&box HUDs default to `pointer-events: none` so input passes through to the game — if your hover transitions aren't firing, this is the first thing to check.

`&` (SCSS parent reference)

Stands in for the enclosing selector when nesting. `&:hover` inside `.HudPanel` compiles to `.HudPanel:hover`. SCSS feature, not CSS — s&box supports it.

How they connect: the shorthand `transition: transform 250ms ease-out 0ms` packs the same information as four longhand declarations. The pseudo-class `(:hover)` defines *when* the property changes; the transition rule defines *how* the change is animated.

2. Longhand form

Lining up the lists

Each of `transition-property`, `-duration`, `-timing-function`, `-delay` accepts a comma-separated list. The values pair up by position: the first duration goes with the first property, second with second, and so on. If one list is shorter, its values repeat.

How they connect: longhand and shorthand compile to the same thing. Use shorthand when you have one property to animate; reach for longhand only when you want different easing per property and the multi-shorthand form (section 3) feels less readable.

3. Multiple transitions in one shorthand

Comma-separated tuples

`transition: transform 200ms ease-out, opacity 400ms ease-in;` is two complete transitions in one declaration. This is the idiomatic way to give each property its own personality — e.g. snap the scale fast while fading the opacity slowly.

How they connect: this is the form you'll write 90% of the time in real s&box UI. It's compact, per-property control, and reads top-to-bottom.

4. Timing functions

`linear`

Constant speed. Mechanical-feeling; correct for progress bars and loaders, wrong for almost everything else.

`ease`

The default if you don't specify. Slight ease-in, stronger ease-out. Equivalent to `cubic-bezier(0.25, 0.1, 0.25, 1.0)`.

`ease-in`

Slow start, fast finish. Best for **exits** — the element accelerates away from the viewer.

`ease-out`

Fast start, slow finish. Best for **entrances** — the element rushes in then settles.

`ease-in-out`

Slow start *and* slow finish. Symmetric. Good for state-to-state morphs where neither end is special (e.g. a card flipping over).

`cubic-bezier(x1, y1, x2, y2)`

A custom curve defined by two control points. The x values must be in `[0, 1]`; the y values can go negative or above 1, producing anticipation (pulling back before moving) or overshoot (going past the target and

settling back). `cubic-bezier(0.34, 1.56, 0.64, 1)` is the classic "back-out" snap that makes toasts and pickups feel satisfying.

How they connect: every named easing is itself a cubic-bezier under the hood. Reach for keyword values first (ease-out for entrances, ease-in for exits); reach for `cubic-bezier()` when you want overshoot or a deliberately weird feel.

5. `:intro` and `:outro` — the s&box-only pseudo-classes

`:intro`

Added by s&box to a panel when it's created, then **removed one frame later**. So whatever values you set under `&:intro` are the *starting* state — the panel transitions *away from them* as soon as the engine strips the class.

`:outro`

Added by s&box when `Panel.Delete()` is called from C#. **The panel is not actually removed until every transition on it has finished**. So you set the *ending* state under `&:outro`, and the engine handles the timing of the actual destruction.

`Panel.Delete()` (C# method)

The s&box API that begins panel removal. Crucially, it does **not** synchronously destroy the panel — it flags it for outro, waits for transitions, then removes it.

How they connect: this pair is the headline s&box feature for UI animation. In web CSS you'd reach for a JS library to coordinate enter/exit. In s&box you just write two SCSS rules and call `Panel.Delete()` — the engine choreographs the rest. Asymmetric exits (different easing, different transforms) are encouraged.

6. State classes — the bread-and-butter pattern

`Panel.SetClass(string name, bool on)` (C# method)

Toggles a CSS class on or off based on a bool. The standard way to drive UI state from C#: every frame (or on events), call `SetClass("selected", isSelected)` and let the SCSS transition handle the visual change.

Why not just re-render?

Razor panels rebuild when `BuildHash()` returns a different value. A rebuild replaces the DOM tree, which **resets any in-flight transitions** on the affected elements. So for things that should animate (hover responses, selection feedback, health changes), keep the element stable and toggle classes rather than conditionally rendering new markup.

How they connect: Razor handles structure (what panels exist); C# `SetClass` handles state (which classes are on); SCSS handles motion (how state changes look). Keep those three layers distinct and your transitions will be reliable.

7. Razor-driven inline style

Inline `style="..."` attribute

A standard HTML escape hatch for putting a CSS declaration directly on an element. In Razor: `style="width: @HealthPercent%;"` interpolates a C# value into the markup at render time.

Why this pattern matters in s&box

s&box does **not** support CSS custom properties (`var(--my-var)`) — confirmed via Facepunch GitHub issue #3043. So the standard web technique of binding C# values to `--variables` doesn't work. The workaround is to write the dynamic value directly into the element's `style` attribute and let the SCSS transition rule on the `class` catch the change.

How they connect: the SCSS class declares *how* to animate (the transition rule); Razor's inline style declares *what* the target value is. When Razor re-renders with a new value, the existing transition rule animates from the old value to the new one.

8. Asymmetric enter/exit

Different transitions per state

The transition rule that's active **at the moment a property changes** is the one that gets used. So you can declare one transition on the base class and a different one on a state class — entering and exiting will then have different speed and easing.

How they connect: real UI almost never wants symmetric motion. A modal opening should feel different from a modal closing. Putting the closing transition on a `.closing` state class (added from C# via `SetClass`) gives you per-direction control with no JavaScript timing.

9. SCSS variables for design tokens

`$variable: value;`

SCSS pre-processor variable. **Resolved at compile time, not at runtime** — you cannot change them from C#. Used for design tokens like consistent durations, easings, and colors across your whole UI.

Why these, not CSS custom properties?

Because s&box doesn't support runtime CSS variables (see section 7). SCSS `$variables` give you the *organizational* benefit (one place to define `$dur-base`) without the *runtime* benefit (changing it on the fly from code).

How they connect: treat `$variables` like constants. They keep your transitions consistent across panels — every button has the same hover duration, every modal the same enter curve — and they cost nothing at runtime.

10. Cost matrix — cheap vs expensive properties

GPU-composited (cheap)

transform and opacity. These don't trigger layout or paint; the GPU handles them as compositor-layer changes. Transition these freely.

Paint-only (medium)

background-color, color, border-color. No layout, but the affected region must repaint. Fine for most UI.

Layout-triggering (expensive)

width, height, top, left, right, bottom, margin, padding, font-size. Each frame of the animation forces s&box to recalculate the position of every sibling and descendant. In a game running at 60+ FPS alongside a 3D scene, this matters.

Rule of thumb

- To **move** an element: transform: translate(), not top/left.
- To **grow/shrink**: transform: scale(), not width/height, **unless** the element genuinely needs to push siblings around (e.g. a healthbar fill, a collapsible drawer).

How they connect: the cheap pair (transform, opacity) handles 80% of UI animation. Reach for layout properties only when you genuinely need siblings to reflow, and accept the cost knowingly.

11. Sound on enter/exit

sound-in / sound-out (s&box custom CSS) ??

Custom CSS properties added by s&box. sound-in plays a sound event when the panel is created; sound-out plays one when it's deleted. The exact value syntax (sound event path vs. sound name) I haven't fully verified — flag and check before relying on it.

How they connect: alongside the visual :intro/:outro transitions, the engine can fire a sound event automatically at the same moments. UI gets a complete sensory polish pass without C# audio code.

12. Things you cannot transition

Discrete property changes

display: none ↔ display: flex, visibility: hidden ↔ visible — these snap instantly. There's no "in-between" value the engine can interpolate to.

auto values

You cannot transition from width: auto to width: 200px (or back). auto resolves at layout time and isn't a number the engine can interpolate.

The "fake hide" pattern

To make an element fade out and become unclickable while staying in the panel tree (so the transition has something to animate), combine three things on a state class: opacity: 0, transform: scale(0.95), and pointer-events: none. The element is visually gone, mouse passes through, but it's still there for the transition.

How they connect: when a transition won't work directly, the workaround is almost always to animate a continuous proxy (opacity, scale) while toggling discrete behavior (pointer-events) at the boundaries.

Mental model in one sentence

A transition declares how a single property should ease between values when it changes, and in s&box you trigger those changes by toggling state classes from C# (SetClass), letting Razor inline-style values, or letting the engine's :intro/:outro pseudo-classes choreograph creation and destruction.

```
// =====  
// CSS transitions in s&box – annotated reference  
// Format: SCSS (Sandbox SCSS subset). Paired with PanelName.razor.  
// =====  
  
// -----  
// 1. The shorthand and the four longhand properties  
// -----  
  
.HudPanel {  
    // property: transition (CSS shorthand) – combines transition-property,  
    //           transition-duration, transition-timing-function, and  
    //           transition-delay into one declaration. Order matters:  
    //           the FIRST <time> is duration, the SECOND <time> is delay.  
    //  
    // value: transform – names which CSS property to animate. Other common  
    //           choices: opacity, background-color, width. Use `all` to  
    //           animate every change (lazy; avoid in production).  
    //  
    // value: 250ms – the duration. Accepts s or ms.  
    //  
    // value: ease-out – timing function (the easing curve).  
    //  
    // value: 0ms – the delay before the transition starts.  
    transition: transform 250ms ease-out 0ms;  
  
    // property: transform – the "starting state" for the transition.
```

```

//      Without this, transitions still work but the implicit
//      start value (translateX(0) here) is what the browser/engine
//      assumes. Being explicit aids readability.
transform: translateX( 0 );

// pseudo-class: :hover – applied when the cursor is over the panel.
//      REQUIRES `pointer-events: all` somewhere up the chain,
//      or it won't fire. Common HUD gotcha. ??
//      (s&b defaults to no pointer events on HUD roots.)
//
// & – SCSS parent selector. `&:hover` here compiles to `.HudPanel:hover`.
&:hover {
    transform: translateX( 16px );
}
}

```

```

// -----
// 2. Longhand form – exactly the same effect, more explicit
// -----

```

```

.HudPanel {
    // property: transition-property – comma-separated list of CSS property
    //      names that should animate when they change. Anything not
    //      listed snaps instantly.
    transition-property: transform, opacity;

    // property: transition-duration – one duration per property in the
    //      list above (or one value applied to all). Match the order.
    transition-duration: 250ms, 400ms;

    // property: transition-timing-function – easing curve per property.
    //      Keyword values: linear, ease, ease-in, ease-out, ease-in-out.
    //      Function value: cubic-bezier(x1, y1, x2, y2). Stepped values
    //      (steps(...), step-start, step-end) probably work too. ??
    transition-timing-function: ease-out, ease-in;

    // property: transition-delay – wait this long after the property
    //      CHANGES before starting the animation. Useful for staggering.
    //      Negative values start mid-animation (advanced).
    transition-delay: 0ms, 100ms;
}

```

```

// -----
// 3. Multiple transitions in one shorthand (comma-separated)
// -----

```

```

.Card {
    // Each comma-separated group is a complete (property, duration,
    // timing-function, delay) tuple. This is the idiomatic way to give
    // each property its own personality – e.g. snappy transform, slow fade.

```



```

transition:
  transform 200ms ease-out 0ms,
  opacity   400ms ease-in 0ms,
  background 600ms linear 100ms;
}

// -----
// 4. Timing functions – what each curve feels like
// -----

.Examples {
  // value: linear – constant speed. Feels mechanical; good for loaders,
  //                progress bars, anything where natural motion is wrong.
  transition-timing-function: linear;

  // value: ease – the default. Slight ease-in, stronger ease-out.
  //                Equivalent to cubic-bezier(0.25, 0.1, 0.25, 1.0).
  transition-timing-function: ease;

  // value: ease-in – slow start, fast finish.
  //                Use for EXITS (element rushing offscreen).
  transition-timing-function: ease-in;

  // value: ease-out – fast start, slow finish.
  //                Use for ENTRANCES (element settling into place).
  transition-timing-function: ease-out;

  // value: ease-in-out – slow start AND slow finish.
  //                Symmetric, good for state-to-state morphs.
  transition-timing-function: ease-in-out;

  // value: cubic-bezier(x1, y1, x2, y2) – custom curve. x must be [0,1];
  //                y can go negative or above 1 to create overshoot / anticipation.
  //                This one is "back-out" – overshoots then settles. Great for
  //                toast pop-ins, pickup notifications, anything you want to feel
  //                bouncy and playful.
  transition-timing-function: cubic-bezier( 0.34, 1.56, 0.64, 1 );
}

// -----
// 5. :intro and :outro – the s&box-only pseudo-classes
// -----
// These do not exist in web CSS. They're s&box's solution to the
// "how do I animate creation/destruction" problem.
//
// :intro – added when the Panel is created, then removed one frame later.
//          So you transition AWAY from the :intro state.
// :outro – added when Panel.Delete() is called in C#. The engine WAITS
//          for all transitions on the panel to finish before actually
//          removing it from the tree. So you transition TOWARDS the

```

```

//      :outro state, and the engine handles the cleanup timing.

.Toast {
  transition: transform 400ms ease-out, opacity 400ms ease-out;

  transform: scale( 1 );
  opacity: 1;

  // pseudo-class: :intro – element is "born" with this class, then
  //                  it's stripped one frame later. The transition runs
  //                  from these values toward the rule's defaults.
  &:intro {
    transform: scale( 0 );
    opacity: 0;
  }

  // pseudo-class: :outro – engine adds this when Panel.Delete() is called.
  //                  The panel won't actually be removed until every
  //                  transition on it has finished. Same idea as the intro
  //                  but in reverse, and you can pick a totally different
  //                  exit style (e.g. fade up while shrinking).
  &:outro {
    transform: scale( 1.4 );
    opacity: 0;
  }
}

// -----
// 6. State classes – the bread-and-butter pattern in s&box
// -----
// Because Razor BuildHash() rebuilds can interrupt in-flight transitions,
// you usually want to toggle a CSS class via Panel.SetClass(name, bool)
// from C# rather than conditionally rendering new elements.
//
// C# side: myPanel.SetClass( "selected", isCurrentlySelected );

.InventorySlot {
  transition: transform 200ms ease-out, border-color 200ms ease-out;

  border: 2px solid rgba( 255, 255, 255, 0.1 );
  transform: scale( 1 );

  // chained class selector – `.InventorySlot.selected`
  // (no `&` needed; this is plain CSS, written as nested for SCSS clarity)
  &.selected {
    border-color: rgba( 255, 220, 0, 1 );
    transform: scale( 1.08 );
  }

  &.disabled {
    opacity: 0.4;
  }
}

```

```

    }
}

// -----
// 7. Razor-driven inline style – for per-instance dynamic values
// -----
// s&box does NOT support CSS custom properties (var(--foo)) ??
// (confirmed via Facepunch GitHub issue #3043).
//
// If you need a value computed in C# to drive an animation, the standard
// pattern is to write the value as an inline style from Razor, and let
// the transition on the class catch the change:
//
// Razor:
// <div class="fill" style="width: @HealthPercent%;" />
//
// SCSS:

.fill {
    // The class declares HOW to animate width; Razor declares WHAT width.
    transition: width 500ms ease-out;
    background-color: #c33;
}

// -----
// 8. Asymmetric enter/exit (the polish move)
// -----
// Real UI rarely uses the same easing in both directions. A common pro
// pattern: snappy ease-out on the way in, slower ease-in on the way out.

.Modal {
    transition: transform 200ms ease-out, opacity 200ms ease-out;

    transform: scale( 1 );
    opacity: 1;

    // When the modal is closing (state class added from C#), use a
    // different transition. The browser/engine picks the transition
    // that's active AT THE MOMENT THE PROPERTY CHANGES.
    &.closing {
        transition: transform 350ms ease-in, opacity 350ms ease-in;
        transform: scale( 0.95 );
        opacity: 0;
    }
}

// -----
// 9. SCSS variables – the right way to share values
// -----

```

```
// Since CSS custom properties don't work, fall back on SCSS variables.  
// These are resolved AT COMPILE TIME, not at runtime – you can't change  
// them from C#. They're for design tokens (consistent timings, colors).
```

```
$dur-fast: 150ms; // pre-processor variable: substituted before  
$dur-base: 250ms; // the SCSS is sent to the engine  
$dur-slow: 400ms;  
$ease-snap: cubic-bezier( 0.34, 1.56, 0.64, 1 );
```

```
.Button {  
  transition:  
    transform $dur-fast $ease-snap,  
    background-color $dur-base ease-out;  
  
  &:hover {  
    transform: translateY( -2px );  
  }  
}
```

```
// -----  
// 10. Cost matrix – what's cheap and what makes the engine cry  
// -----  
// In a game UI running alongside a 3D scene, every layout-triggering  
// transition costs you frame budget.  
//  
// CHEAP (GPU-composited): transform, opacity  
// OK (paint, no layout): background-color, color, border-color  
// EXPENSIVE (forces layout): width, height, top, left, right, bottom,  
// margin, padding, font-size  
//  
// Rule: if you want an element to "move", prefer transform: translate()  
// over animating top/left. If you want it to "grow", prefer  
// transform: scale() over animating width/height – UNLESS the element  
// genuinely needs to push siblings around, in which case animating  
// width is correct (e.g. a healthbar fill in its own row).
```

```
// -----  
// 11. Sound on enter/exit (s&box custom CSS) ??  
// -----  
// s&box has custom CSS properties for playing a sound event when a panel  
// is created or deleted. Confirmed pair: sound-in, sound-out.  
// (Names verified in the s&box sound docs. Format and supported value  
// syntax – sound event paths vs sound names – I haven't fully verified;  
// flagging with ??.)
```

```
.NotifyPopup {  
  sound-in: "ui.notify.open"; // played when panel appears ??  
  sound-out: "ui.notify.close"; // played when panel is deleted ??  
}
```

```

// -----
// 12. Things you CANNOT transition
// -----
// - display: none ↔ flex / block (snaps instantly)
// - Anything between `auto` and a concrete length (auto isn't interpolable)
// - Layout-mode changes (e.g. flex-direction)
// - Non-numeric values without a defined "in-between" (font-family, etc.)
//
// To fake hiding/showing, animate opacity + scale and set pointer-events
// based on the same state class:

.Hidable {
  transition: opacity 200ms ease-out, transform 200ms ease-out;

  opacity: 1;
  transform: scale( 1 );
  pointer-events: all;

  &.hidden {
    opacity: 0;
    transform: scale( 0.95 );
    // property: pointer-events – controls whether the panel receives
    // mouse input. Setting to `none` here means a "hidden"
    // element can't be accidentally clicked while it's
    // fading out, but it stays in the panel tree so the
    // transition has something to animate.
    pointer-events: none;
  }
}

```